

Conception Architecturale d'un Système Multi-Agents pour l'Analyse Multi-Perspectives de Comptes Rendus de Réunions

Bensmina Anass & Moubarak Benaqqa
École Nationale Supérieure d'Informatique
et d'Analyse des Systèmes (ENSIAS)
Université Mohammed V, Rabat, Maroc

Résumé—Ce rapport présente la conception architecturale d'un système multi-agents (SMA) dédié à l'analyse automatisée de comptes rendus de réunions. Le système repose sur trois agents analyseurs spécialisés, chacun portant une perspective distincte sur la même transcription, coordonnés par un agent de consensus chargé de fusionner leurs sorties et de résoudre les désaccords. L'ensemble est orchestré par un pipeline à deux étapes et s'appuie sur une mémoire partagée persistante en SQLite. L'architecture vise à démontrer qu'un collectif d'agents légers et spécialisés, tournant sur un modèle de langage modeste (Qwen 2.5 7B), peut produire une analyse plus riche et plus fiable qu'un agent unique généraliste. L'implémentation cible s'appuie sur le framework CrewAI.

Index Terms—système multi-agents, analyse de réunions, CrewAI, consensus, exécution parallèle, Qwen

I. INTRODUCTION

L'analyse manuelle des comptes rendus de réunions est une tâche coûteuse et sujette aux erreurs. Les décisions prises, les tâches assignées et les risques évoqués lors de réunions successives forment un corpus évolutif dont la cohérence doit être maintenue. Or, une seule personne — ou un seul modèle de langage — qui lit une transcription apporte inévitablement une perspective unique, avec ses angles morts.

Les systèmes multi-agents offrent une réponse naturelle à cette limitation : plutôt que de confier l'ensemble de l'analyse à un seul agent, on la distribue entre plusieurs agents spécialisés qui examinent la même transcription sous des angles complémentaires. Leurs sorties, parfois convergentes et parfois contradictoires, sont ensuite réconciliées par un mécanisme de consensus.

La question centrale de ce projet est la suivante : un système multi-agents composé de petits modèles spécialisés peut-il produire une analyse de meilleure qualité qu'un modèle unique généraliste opérant seul ? L'hypothèse est que l'architecture — la spécialisation des perspectives, le parallélisme et le consensus — compte davantage que la taille du modèle.

Le domaine d'application retenu est celui d'une startup B2B SaaS fictive, avec un corpus de dix transcriptions synthétiques contenant des éléments volontairement ambigus, des contradictions plantées et des chevauchements entre dimensions d'analyse.

II. PROPRIÉTÉS SMA CIBLÉES

Le système est conçu pour illustrer cinq propriétés fondamentales des systèmes multi-agents.

La **spécialisation** signifie que chaque agent possède un rôle distinct, un prompt dédié et un schéma de sortie propre. Aucun agent ne tente de tout couvrir ; chacun est optimisé pour sa perspective.

L'**exécution parallèle** permet aux trois agents analyseurs de traiter la même transcription simultanément, réduisant ainsi la latence globale du pipeline par rapport à une exécution séquentielle.

La **mémoire partagée** offre un dépôt commun dans lequel les résultats de chaque réunion sont stockés et accessibles à toutes les analyses futures, permettant un suivi longitudinal.

Le **raisonnement inter-temporel** exploite cette mémoire pour détecter des incohérences entre les réunions : une décision prise aujourd'hui qui contredit celle de la semaine passée, un délai qui a changé sans explication.

Enfin, la **résolution de conflits** intervient lorsque les agents analyseurs produisent des interprétations divergentes du même élément. Le mécanisme de consensus tranche, pondère et produit une synthèse unifiée.

III. ARCHITECTURE ET RÔLE DES AGENTS

Le système adopte une topologie en étoile : les agents n'interagissent jamais directement entre eux. Toute communication passe par le pipeline d'orchestration, ce qui garantit un point unique de contrôle et d'observabilité.

A. Les Trois Agents Analyseurs

Les trois agents reçoivent la même entrée — la transcription brute accompagnée de ses métadonnées — et l'examinent chacun sous un angle différent. C'est précisément ce chevauchement de périmètre qui rend le consensus pertinent : un même élément de la transcription peut apparaître dans les sorties de deux ou trois agents, mais avec un cadrage différent.

L'**analyseur d'actions** se concentre sur la dimension opérationnelle : qui s'est engagé à faire quoi, pour quand, et avec quelles dépendances. Il cherche les tâches concrètes, les propriétaires, les échéances et les liens entre actions. Sa question directrice est : « qui fait quoi, et quand ? »

L'**analyseur stratégique** se concentre sur la dimension décisionnelle : quelles orientations ont été prises, quelles priorités ont changé, quels compromis ont été acceptés et pourquoi. Il cherche les décisions, les pivots, les arbitrages et leur justification. Sa question directrice est : « où va-t-on, et pourquoi ? »

L'**analyseur de risques** se concentre sur ce qui pourrait mal tourner : les questions restées sans réponse, les responsabilités non assignées, les délais irréalistes, les contradictions avec les réunions passées et les blocages potentiels. Sa question directrice est : « qu'est-ce qui peut échouer ? »

Ces trois agents s'exécutent en parallèle, ce qui réduit la latence par rapport à un traitement séquentiel.

B. L'Agent de Consensus

L'agent de consensus intervient en seconde phase. Il reçoit les trois sorties des analyseurs et produit un rapport unifié. Son travail comporte trois volets.

Premièrement, il identifie les **convergences** : lorsque deux ou trois agents signalent le même élément, la confiance dans cet élément est élevée. Deuxièmement, il identifie les **divergences** : lorsque les agents attribuent des propriétaires différents à la même tâche, ou qualifient le même événement différemment, le consensus signale le conflit et propose une résolution pondérée. Troisièmement, il assure le **raisonnement inter-temporel** en interrogeant la mémoire partagée pour comparer les extractions courantes avec l'historique des réunions précédentes, détectant ainsi les contradictions inter-temporelles.

Le consensus produit un rapport final dans lequel chaque élément est accompagné d'un score de confiance reflétant le degré d'accord entre les agents.

C. L'Orchestration

L'orchestration n'est pas un agent au sens intelligent du terme. C'est du code Python qui gère le flux du pipeline : ingérer la transcription, lancer les trois analyseurs en parallèle, collecter leurs résultats, les transmettre à l'agent de consensus, puis écrire le rapport final en mémoire. Dans CrewAI, cela correspond à deux objets Crew enchaînés : le premier exécute les trois tâches d'analyse en parallèle, le second lance la tâche de consensus sur les résultats fusionnés.

IV. MÉMOIRE PARTAGÉE

La mémoire partagée est implémentée sous forme d'une base SQLite locale. Ce choix est motivé par la simplicité : aucun serveur à déployer, aucune dépendance externe, et des capacités de requêtage suffisantes pour le volume de données concerné (dix réunions).

La base stocke l'ensemble des actions, décisions, risques et résultats de consensus extraits au fil des réunions. Chaque entité est liée à sa réunion d'origine et peut référencer d'autres entités par identifiant. Les décisions sont marquées comme actives, remplacées ou révoquées. Les tâches conservent un historique de statut. Les risques enregistrent leur évolution.

Pour le raisonnement inter-temporel, l'agent de consensus a besoin de retrouver les éléments historiques pertinents par

similarité de sujet. Plutôt que d'intégrer une base vectorielle dédiée, la similarité est calculée en Python via TF-IDF et cosinus sur les descriptions textuelles des éléments. À l'échelle de dix réunions, cette approche est suffisante et n'ajoute aucune infrastructure.

V. STACK TECHNIQUE

Le choix de la stack est guidé par un principe : minimiser l'infrastructure tout en maximisant la démonstration des propriétés SMA.

Le **framework d'orchestration** retenu est CrewAI, qui permet de définir des agents avec des rôles, des objectifs et des backstories distincts, et de les coordonner via des tâches séquentielles ou parallèles. Les contrats de sortie sont garantis par des schémas Pydantic.

Le **modèle de langage** est Qwen 2.5 7B Instruct, quantifié en Q4 et servi localement via llama-server avec le flag `-parallel 6`. Ce choix permet l'inférence concurrente de six requêtes sur le même modèle chargé en mémoire, pour une empreinte d'environ 4,5 Go de RAM. Qwen 2.5 7B a été retenu pour sa conformité supérieure aux instructions structurées et sa fiabilité en génération JSON, critères essentiels pour les contrats de sortie Pydantic.

La **mémoire partagée** repose sur SQLite, avec une couche de similarité textuelle en TF-IDF calculée en Python. Aucune base vectorielle externe n'est nécessaire.

Le **corpus** est constitué de dix transcriptions synthétiques au format JSON, chacune contenant les métadonnées de la réunion (date, participants, ordre du jour) et le texte brut de la transcription.

VI. ÉVALUATION PRÉVUE

L'évaluation vise à répondre à la question centrale : le système multi-agents fait-il mieux qu'un agent unique ?

Pour cela, un **agent baseline** sera construit : un seul agent généraliste, utilisant le même modèle Qwen 2.5 7B, qui analyse chaque transcription en un seul passage. Les deux systèmes seront comparés sur l'ensemble du corpus selon plusieurs métriques.

Le **temps d'exécution** comparera la latence du pipeline parallèle à celle de l'exécution séquentielle et du baseline mono-agent. Le **taux de désaccord** mesurera la proportion d'éléments sur lesquels les analyseurs divergent, indicateur de la richesse apportée par la multiplicité des perspectives. La **cohérence intra-agent** vérifiera que chaque analyseur reste fidèle à sa perspective sans dériver vers le périmètre des autres. La **valeur ajoutée du consensus** quantifiera les cas où le consensus enrichit, corrige ou inverse la sortie d'un agent individuel. Enfin, une **comparaison avec un tri humain** sur un sous-échantillon de trois à quatre transcriptions fournira un point de référence externe.

Des tables d'exemples convergents et divergents illustreront concrètement les résultats.

VII. LIMITES ET PERSPECTIVES

Plusieurs limites sont anticipées. La taille du modèle (7B paramètres) impose des prompts très structurés et laisse peu de marge pour le raisonnement nuancé ; les sorties JSON mal-formées nécessiteront un mécanisme de fallback. Le périmètre de chevauchement entre agents est défini par leurs prompts, ce qui signifie que la qualité du consensus dépend directement de la qualité de l'ingénierie de prompts. Enfin, la similarité TF-IDF est un proxy limité pour la recherche sémantique inter-temporelle ; un corpus plus large justifierait une base vectorielle.

Les perspectives d'amélioration incluent l'ajout d'un agent critique capable de remettre en question les sorties des autres agents avant le consensus, une pondération adaptative des perspectives en fonction du type de réunion, l'intégration de métriques d'équité (fairness) dans l'évaluation, et un mode d'explicabilité permettant de tracer chaque élément du rapport final jusqu'à son origine dans la transcription.

VIII. CONCLUSION

Ce rapport a présenté la conception architecturale d'un système multi-agents pour l'analyse multi-perspectives de comptes rendus de réunions. L'architecture repose sur trois analyseurs spécialisés opérant en parallèle, un agent de consensus et une mémoire partagée en SQLite, le tout orchestré par CrewAI et alimenté par un modèle local Qwen 2.5 7B.

L'hypothèse centrale est que l'architecture — trois perspectives spécialisées plus consensus — produit une analyse plus riche qu'un modèle unique, et que cette richesse vaut davantage qu'un modèle plus gros opérant seul. L'implémentation et l'évaluation sur le corpus synthétique feront l'objet des travaux à venir.

RÉFÉRENCES

- [1] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. John Wiley & Sons, 2009.
- [2] CrewAI, "CrewAI Documentation," 2024. [En ligne]. Disponible : <https://docs.crewai.com>
- [3] J. Ferber, *Multi-Agent Systems : An Introduction to Distributed Artificial Intelligence*. Addison-Wesley, 1999.
- [4] Qwen Team, "Qwen2.5 : A Party of Foundation Models," 2024. [En ligne]. Disponible : <https://qwenlm.github.io/blog/qwen2.5/>
- [5] G. Gerganov, "llama.cpp," 2024. [En ligne]. Disponible : <https://github.com/ggerganov/llama.cpp>